



UNIFACS

Feira de Santana

Noberto Maciel

noberto.maciел@ulife.com.br

Oficina: Processamento de Linguagem Natural

TechWeek XII
UNIFACS 2026.2

- Breve discussão sobre Neurônio Artificial e RNAs
- Features, Descritores e o Espaço de Características
- Neurônio Perceptron
- Redes Convolucionais (RNN)
- Técnicas para Processamento de Linguagem Natural
- Prática com algoritmos em Python

Neurônio Básico e Redes Neurais Artificiais

O que é:

Uma **Rede Neural Artificial (RNA)** é um modelo computacional inspirado no cérebro humano. As RNAs possuem capacidade de aquisição, manutenção e uso do conhecimento.

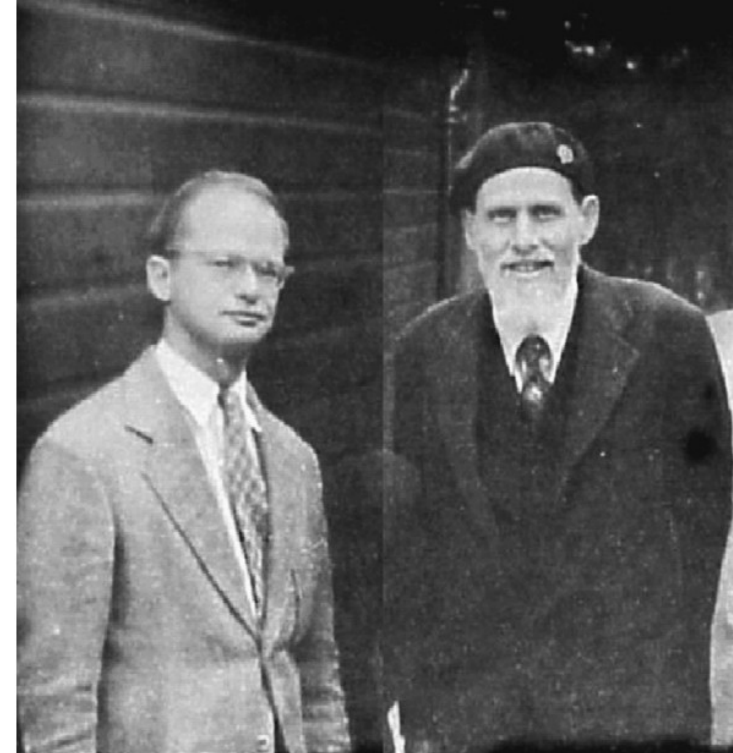
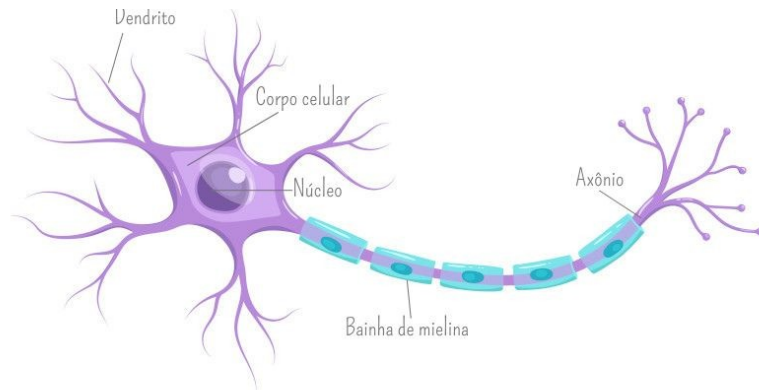
Modelos computacionais distribuídos, compostos por um conjunto de unidades de processamento (**neurônios**), dispostas em uma ou mais camadas, e que são interligadas por diversas conexões (sinapses)

Objetivo:

- Reconhecimento de padrões;
- Categorização (classificação) ou Clustering (agrupamento);
- Aproximação de funções.

O primeiro neurônio artificial foi proposto por Warren McCulloch (neurocientista) e Walter Pitts (lógico autodidata) em 1943.

Os neurônios foram inspirados no funcionamento do cérebro humano e simulavam portas lógicas.



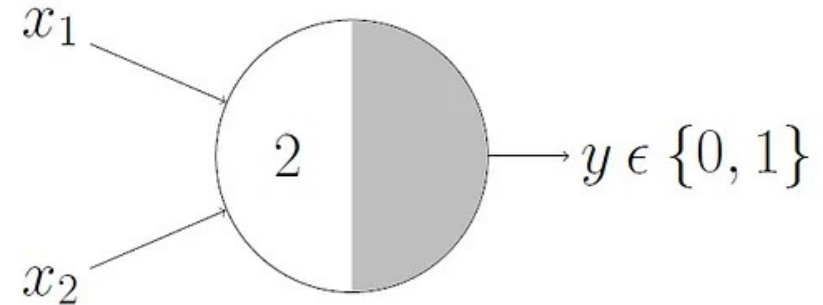
Neurônio básico de McCulloch e Pitts:

A atividade deste neurônio é binária;

O disparo do neurônio deve ocorrer com um certo número de sinapses (impulsos na entrada);

A atividade numa sinapse inibitória impede o disparo do neurônio no instante associado;

A estrutura da rede de neurônios não se altera com o tempo.



AND function

$$x_1 + x_2 = \sum_{i=1}^2 x_i \geq 2$$

Neurônio básico de McCulloch e Pitts:

A atividade deste neurônio é binária;

O disparo do neurônio deve ocorrer com um certo número de sinapses (impulsos na entrada);

A atividade numa sinapse inibitória impede o disparo do neurônio no instante associado;

A estrutura da rede de neurônios não se altera com o tempo.

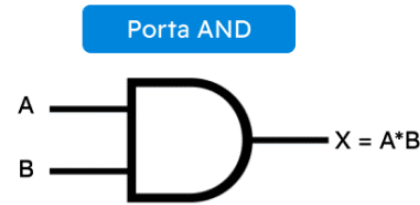
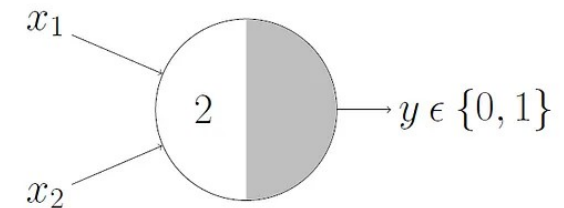
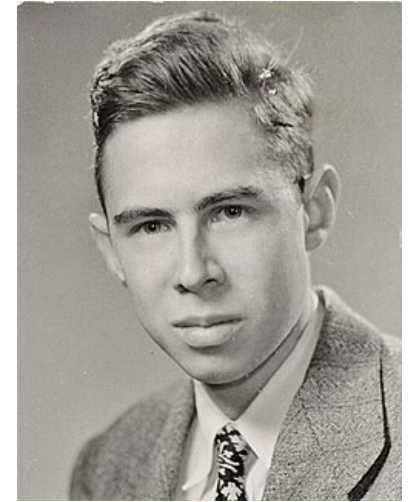


Tabela da Verdade		
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1



- O Perceptron foi proposto pelo psicólogo americano **Frank Rosenblatt** em 1958;
- É composto pelo neurônio de **McCulloch-Pitts**, com função de ativação degrau (limiar) ou sinal (degrau bipolar) e aprendizado supervisionado;
- Possui uma única camada neural, com apenas 1 neurônio;
- Princípio de Aprendizado de **Hebb** (regra de Hebb, postulado de Hebb);



Neurônio artificial Perceptron básico:

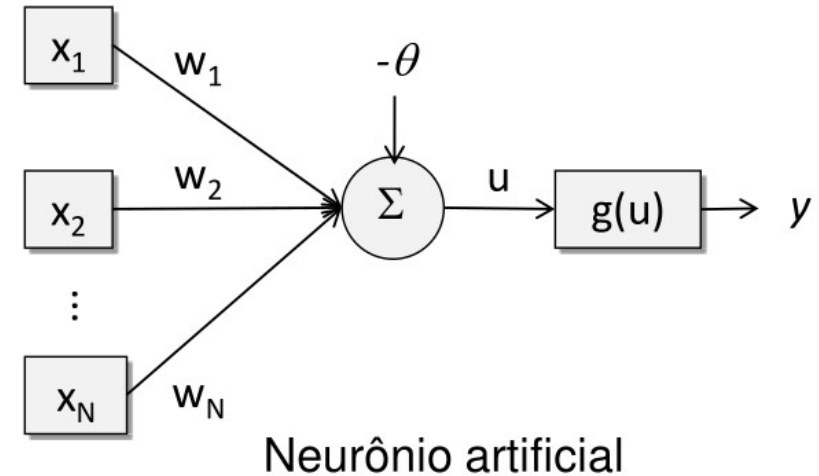
- a) Classificador, identificador de padrões;
- b) Divide o espaço, linearmente, em duas regiões através de um reta:

$$y = ax + b$$

onde: a = coeficiente angular da reta
 b = coeficiente linear

Os pesos são ajustados à cada rodada de treinamento:

$$W_{i\text{-final}} = W_{i\text{-inicial}} + \eta(Y_{\text{real}} - Y_{\text{predito}}) * X_i$$



$$u = \sum_{i=1}^N w_i \cdot x_i - \theta$$

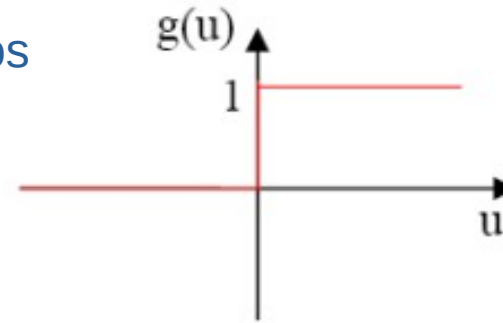
$$y = g(u)$$

A função de ativação processa o conjunto de entradas recebidas e o transforma em um estado de ativação;

Normalmente, o estado de ativação dos neurônios pode assumir os seguintes valores:

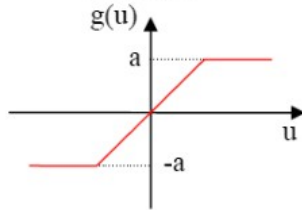
- I) Binários (0 e 1)
- II) Bipolares (-1 e 1)
- III) Reais ($-1 \leq g(u) \leq 1$) ou ($0 \leq g(u) \leq 1$)

- Degrau ou Limiar



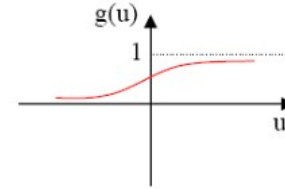
$$g(u) = \begin{cases} 1, & u \geq 0 \\ 0, & u < 0 \end{cases}$$

- Rampa



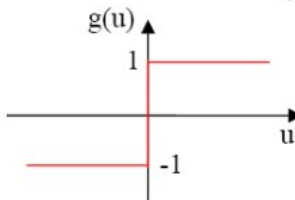
$$g(u) = \begin{cases} a, & \text{se } u > a \\ u, & \text{se } -a \leq u \leq a \\ -a, & \text{se } u < -a \end{cases}$$

- Logística ou Sigmóide



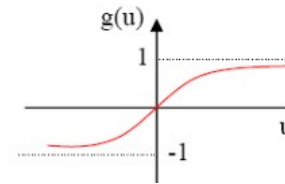
$$g(u) = \frac{1}{1 + e^{-\beta u}}$$

- Sinal ou Degrau Bipolar

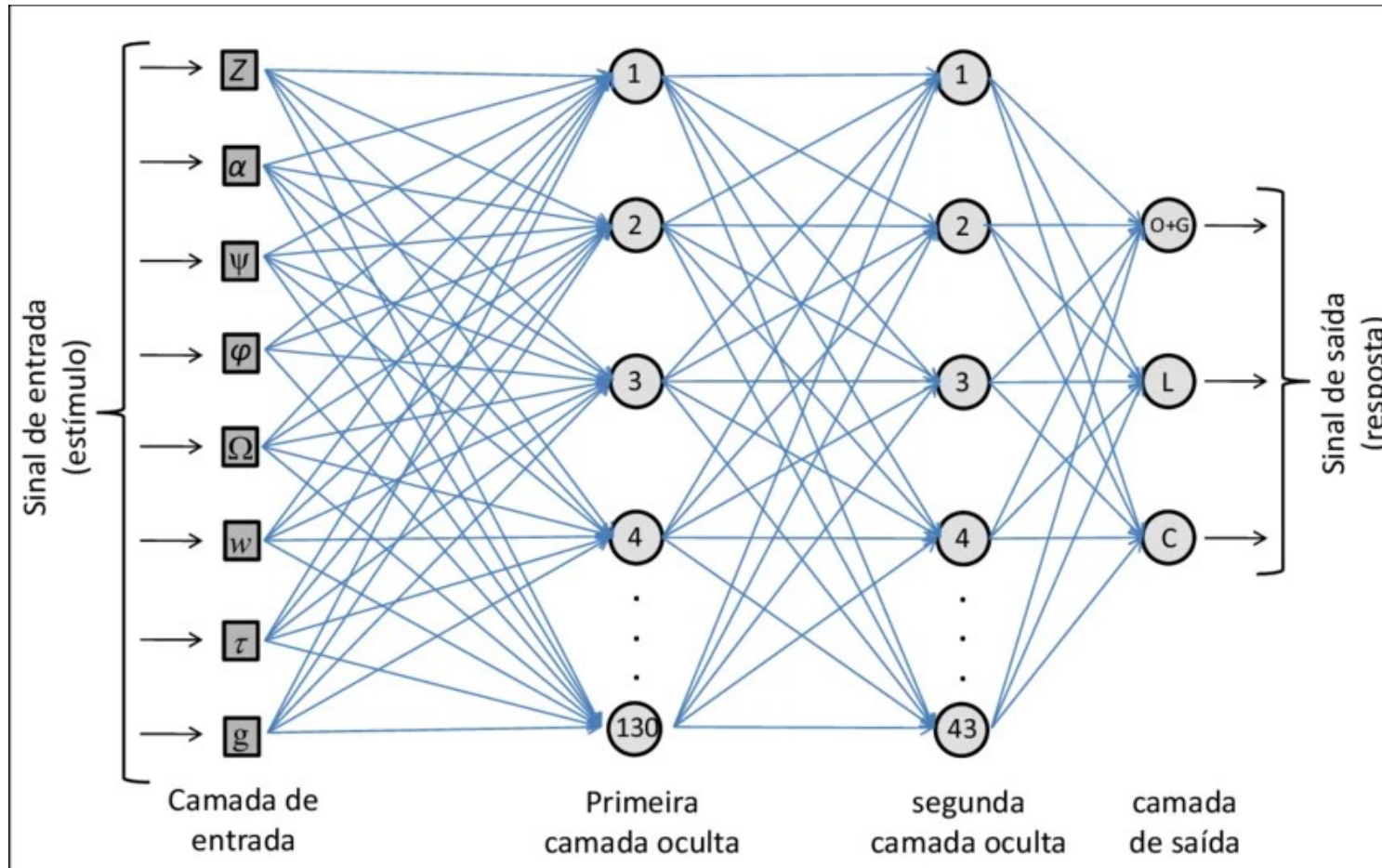


$$g(u) = \begin{cases} 1, & \text{se } u > 0 \\ 0, & \text{se } u = 0 \\ -1, & \text{se } u < 0 \end{cases}$$

- Tangente Hiperbólica



$$g(u) = \frac{1 - e^{-\beta u}}{1 + e^{\beta u}}$$



Fase de treinamento

As redes neurais artificiais aprendem por meio do ajuste iterativo dos pesos de cada neurônio:

1. Vetores de características são apresentados como entrada da rede;
2. A saída gerada é comparada com o valor esperado (rótulo);
3. Se a saída estiver correta, nenhum ajuste é feito para aquela amostra;
4. Se estiver incorreta, calcula-se o erro e os pesos são ajustados;
5. Esse processo é repetido para todas as amostras do dataset, formando uma época;
6. O treinamento geralmente ocorre por várias épocas, até atingir um critério de parada (ex: erro mínimo ou número máximo de iterações).

Fase de teste

Aqui avaliamos o nosso modelo treinado (com pesos ajustados, fixos):

1. Vetores de características não vistos durante o treinamento são apresentados à rede;
2. A saída gerada é comparada com o valor esperado (rótulo);
3. Calculamos métricas de desempenho com base nos acertos e erros;
4. A acurácia mede a proporção de previsões corretas;
5. Também podemos analisar métricas mais detalhadas baseadas em: *verdadeiros positivos (VP)*, *verdadeiros negativos (VN)*, *falsos positivos (FP)*, *falsos negativos (FN)*
6. Essas métricas permitem calcular, por exemplo: *Precision*, *Recall*, *F1-Score* ou *F1-Measure*

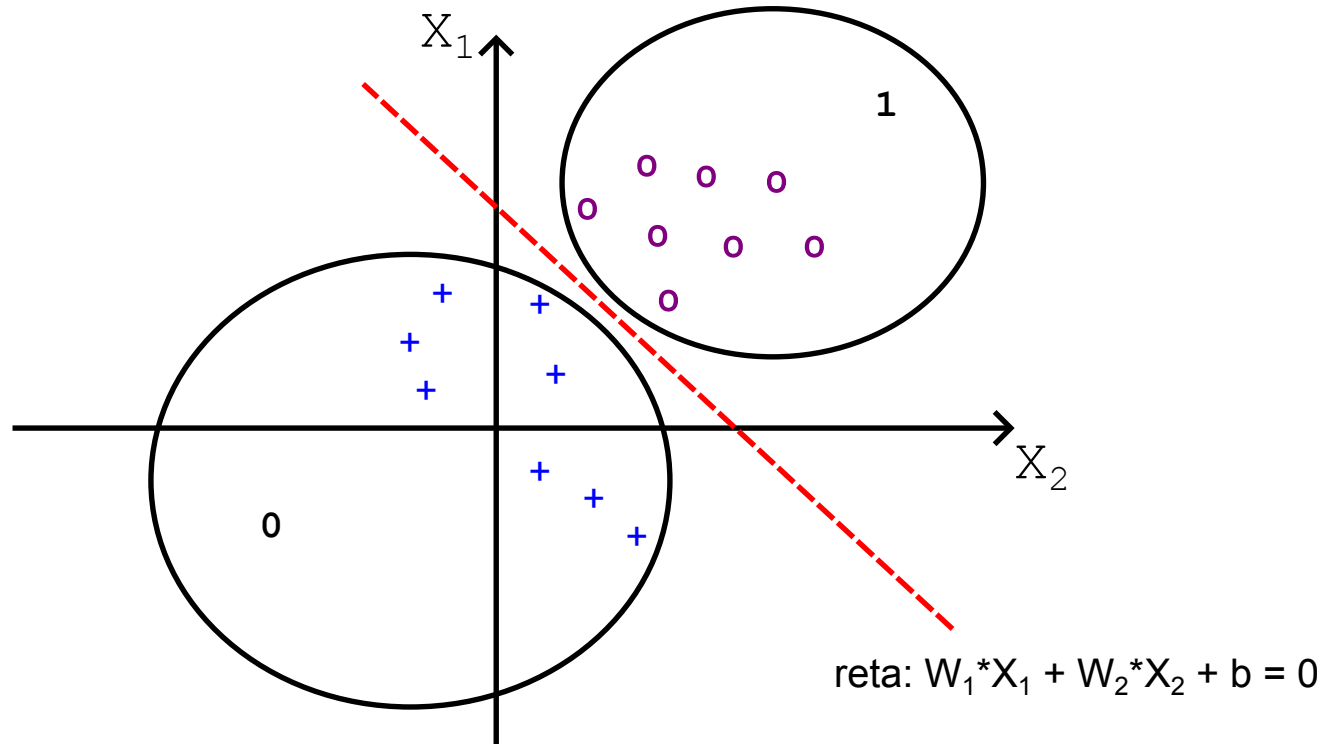
Features, Descritores e o Espaço de Características

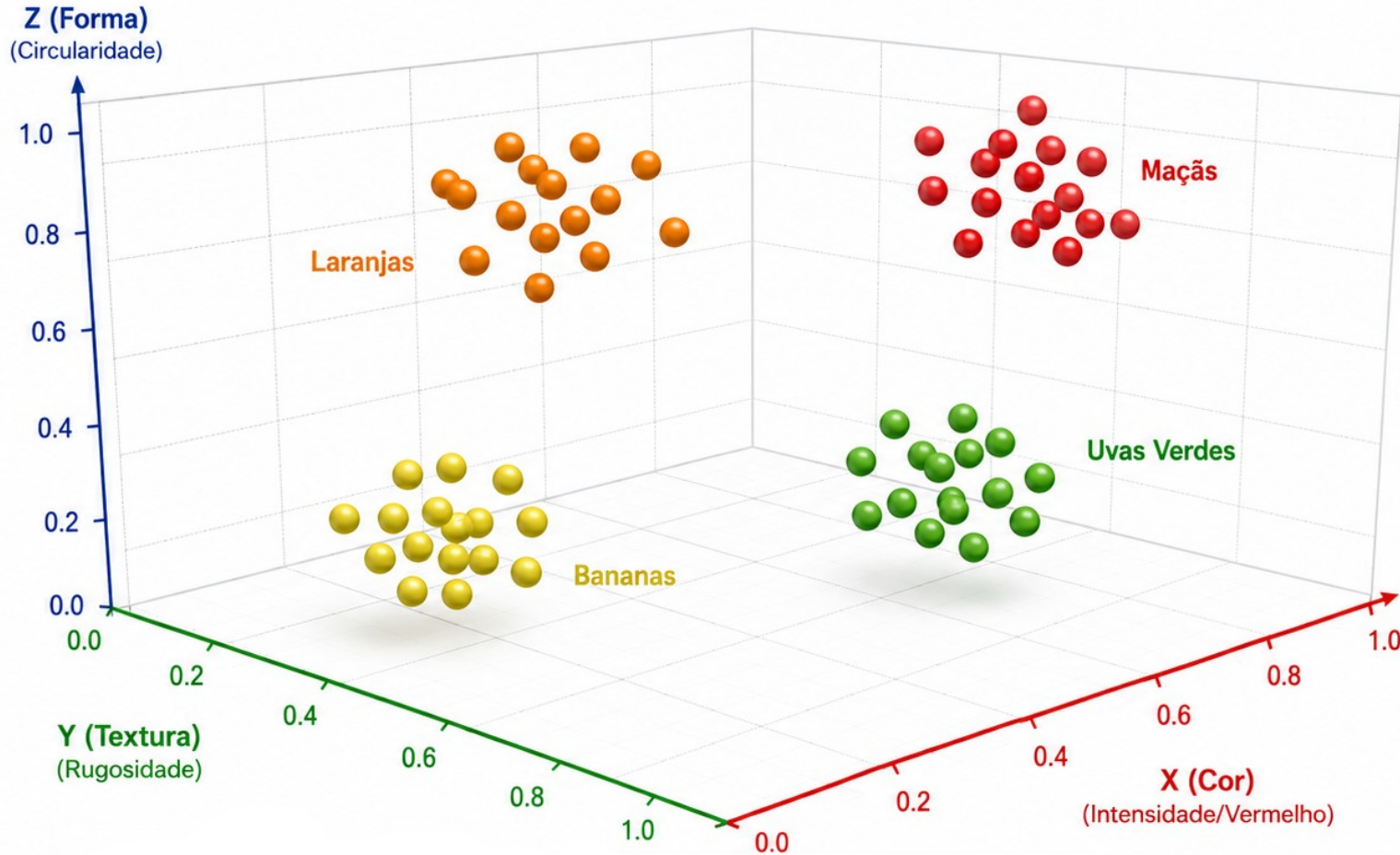
Descritores são representações numéricas, normalmente na forma de **vetores**, que armazenam os valores das características (features) **extraídas de um objeto** por meio de **algoritmos** ou **cálculos matemáticos**.

É possível extrair características de diferentes tipos de dados, como **imagens**, **vídeos**, **áudio** e **texto**.

A dimensão de um descritor corresponde à quantidade de características que ele representa. Assim, se um algoritmo extrai 128 características, o descritor será um vetor de 128 dimensões.

Espaço de características linearmente separável





Características

-  **Forma (Z)**
Circularidade
0 = alongado
1 = circular
-  **Textura (Y)**
Rugosidade
0 = lisa
1 = rugosa
-  **Cor (X)**
Intensidade de Vermelho
0 = pouco vermelho
1 = muito vermelho

Exemplos de Frutas

-  **Maçã**
-  **Laranja**
-  **Banana**
-  **Uva Verde**

Redes Neurais Recorrentes

Recurrent Neural Network (RNN)

As **Redes Neurais Recorrentes** são um tipo de rede que tem a característica de utilizar dados dos **inputs anteriores** para **influenciar** as entradas e saídas **atuais**.

Isso funciona como uma espécie de **memória**, onde os dados anteriormente apresentados fazem parte dos novos resultados, criando a recorrência.

São comumente utilizadas com **dados sequenciais** e **séries temporais**.

Dados sequenciais: são dados organizados em uma ordem específica, onde a posição importa. Cada elemento da sequência depende (ou pode depender) dos anteriores.

Ex.: texto (frase = sequência de palavras); áudio (sequência de amplitudes ao longo do tempo); DNA (sequência de bases genéticas).

Séries temporais: são um tipo específico de dado sequencial em que os dados são coletados em intervalos regulares de tempo. Têm marcadores temporais, podem exibir tendências, sazonalidade, ciclos, etc.

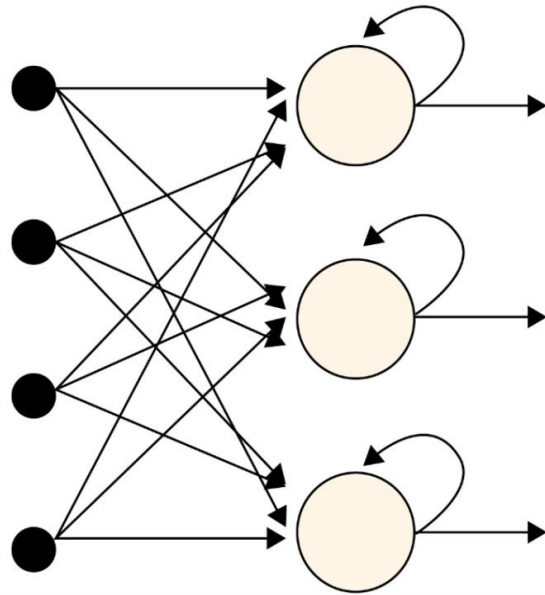
Ex.: cotação de uma moeda, produto ou investimento; leitura de um sensor.

Nas redes **FeedForward** (redes neurais tradicionais), cada conexão entre os nós possui **pesos distintos**.

Já nas **RNNs**, os **pesos** são **compartilhados** ao longo do tempo dentro de uma **mesma camada**.

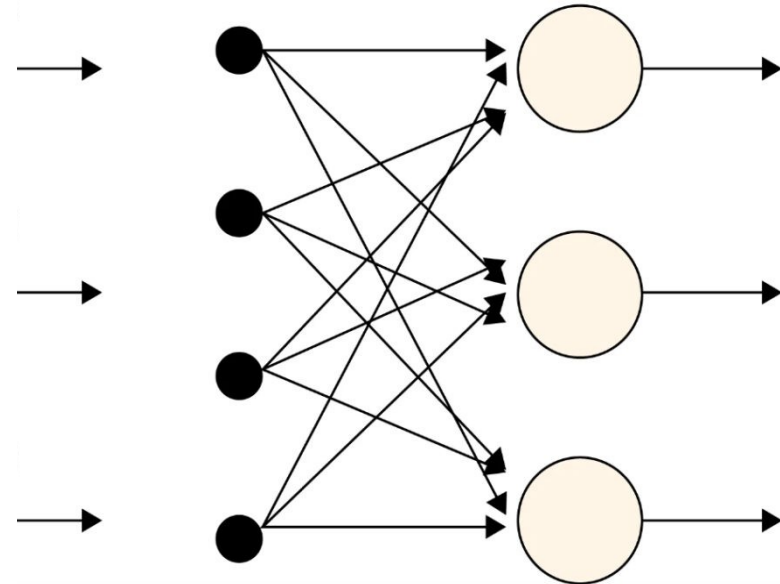
Durante o processo de descida do gradiente, esses **pesos** compartilhados são **ajustados** com base em cada passo da sequência, com o objetivo de **minimizar** a função de **perda**.

Recorrente



VS

Feedforward



Como é feito o treinamento?

A **RNN** é treinada com dados sequenciais. Em alguns casos precisamos de transformar dados categóricos em numéricos, esse método é chamado de **encoding**.

Existem vários métodos, mas destacaremos o **one-hot** e o **embedding encoding**. No método **one-hot** substituímos o conteúdo categórico por **0** ou **1** e formamos uma sequência de bits.

Ex.: Maria está ouvindo música

Temos 4 valores categóricos, então, vamos criar uma sequência com 4 colunas.

One-hot encoding:

Label
Maria
Está
Ouvindo
Música

Maria	Está	Ouvindo	Música
0	0	0	1
0	0	1	0
0	1	0	0
1	0	0	0

A sequência codificada é inserida nas entradas da RNN como um vetor (0,0,0,1).

Embedding encoding:

As palavras (ou tokens) recebem valores numéricos inteiros únicos que são utilizados para dar entrada na camada de **embedding**.

Label	Valor
Está	1
Maria	2
Música	3
Ouvindo	4

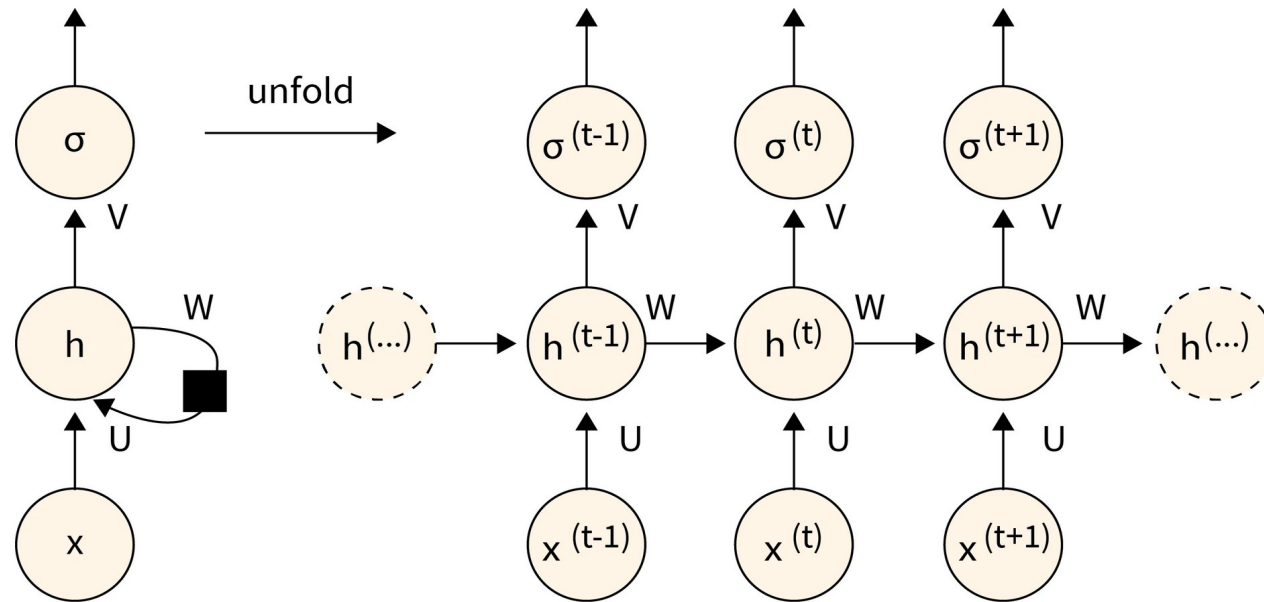
Durante o treinamento com **dados textuais**, a **RNN** processa vetores gerados pela camada de **embedding**, que aprende uma matriz de vetores densos (pesos com diferentes de zero) em que cada vetor representa semanticamente uma palavra. Palavras com significados ou contextos semelhantes acabam tendo vetores próximos nesse espaço, permitindo que a rede capture relações semânticas de forma implícita.

Como é feito o ajuste dos pesos?

A **RNN** utiliza o mesmo mecanismo de uma rede neural tradicional, o **Backpropagation**, porém, com alguns ajustes (Backpropagation Through Time – BPTT).

Como a **RNN** **processa sequências**, ela precisa manter uma **memória** do que já viu. Isso é feito repetindo (ou desenrolando) a rede no tempo, criando uma **cadeia** de cópias da mesma rede para cada passo da sequência.

Os **pesos são os mesmos** durante todos os tempos t de treinamento, o ajuste é feito somente após o cálculo do gradiente descendente no **BPTT**.



x: entrada do neurônio

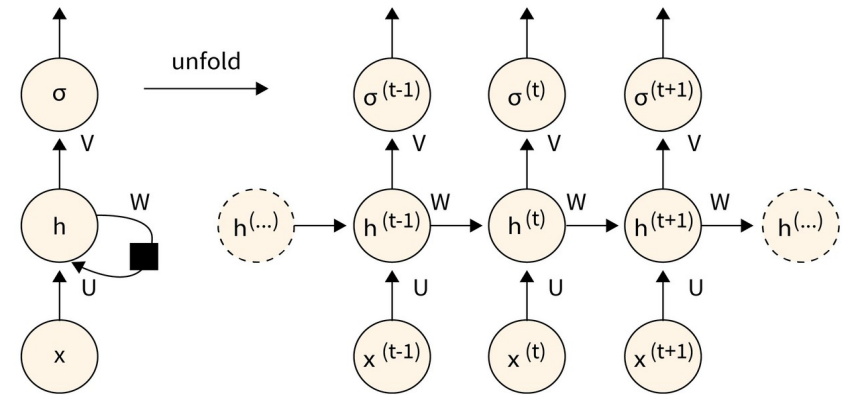
U: peso da entrada para o estado oculto

W: peso de um estado oculto anterior para o atual (recorrente)

V: peso do estado oculto para a saída

h: estado oculto

σ : função de ativação da saída



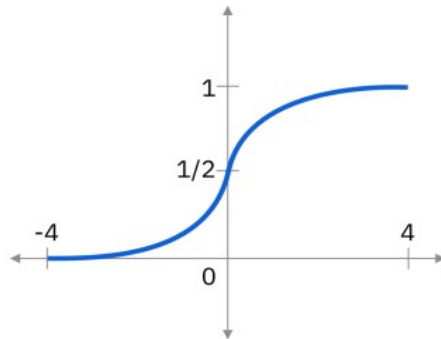
$$h^{(t)} = f(U.x^{(t)} + W.h^{(t-1)} + b_h)$$

$$\sigma^{(t)} = f(V.h^{(t)} + b_\sigma)$$

Funções de ativação

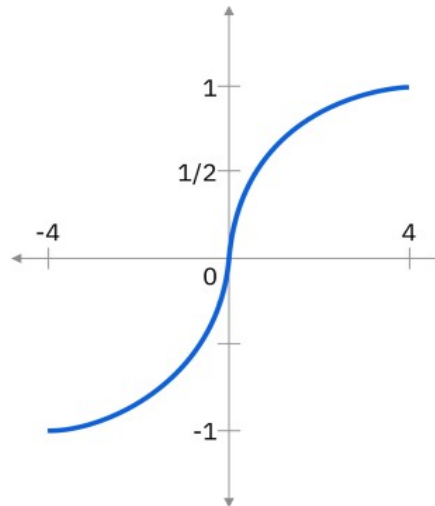
sigmoid

$$g(x) = \frac{1}{1+e^{-x}}$$



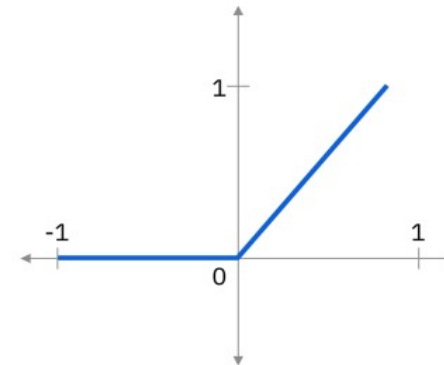
**tangente
hiperbólica (tanh)**

$$g(x) = \frac{e^x - e^{-x}}{1 + e^{-x}}$$



ReLU

$$g(x) = \max(0, x)$$

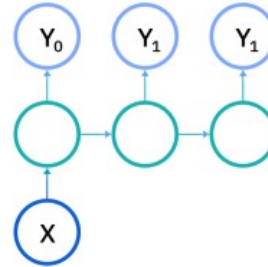


Tipos de RNNs

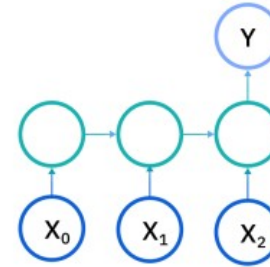
One-to-one



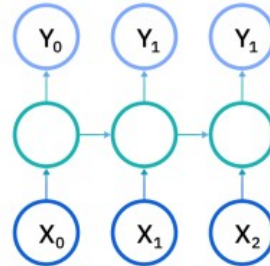
One-to-many



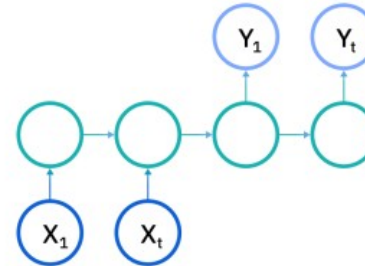
Many-to-one



Many-to-many



Many-to-many



Tipos de RNN:

Redes neurais recorrentes bidirecionais (BRNNs): Enquanto as RNNs unidirecionais só utilizam entradas anteriores para prever o estado atual, as BRNNs incorporam dados futuros para aumentar a precisão da previsão;

Memória de curto longo prazo (LSTM): LSTM é uma arquitetura popular de RNN, desenvolvida por Sepp Hochreiter e Juergen Schmidhuber para solucionar o problema do gradiente desaparecendo. Esse modelo resolveu a questão das dependências de longo prazo;

Unidades recorrentes fechadas (GRUs): Uma GRU (Gated Recurrent Unit) é semelhante à LSTM, pois também foi projetada para resolver o problema da memória de curto prazo presente nas RNNs tradicionais. Diferente das LSTMs, que utilizam um estado de célula separado, a GRU simplifica o modelo ao trabalhar apenas com estados ocultos;

RNNs codificador-decodificador: Elas são amplamente utilizadas em tarefas de transformação de sequências, como a tradução automática. O codificador transforma a sequência de entrada em um vetor de comprimento fixo (contexto), e o decodificador usa esse contexto para gerar a sequência de saída. No entanto, o vetor de contexto com comprimento fixo pode limitar o desempenho, especialmente em sequências de entrada mais longas.

Problemas e limitações:

- Desaparecimento ou aumento exponencial do gradiente devido sua característica de recorrência (**vanishing gradient** e **exploding gradient**);
- Dificuldade de “lembrança” em grandes sequências (quanto maior a sequência, maior o tempo t);
- Treinamento lento;
- Dificuldade em lidar com sequências muito pequenas;
- Dificuldade em lidar com dados desbalanceados ou ruídos nos dados.

Natural Language Processing (NLP)

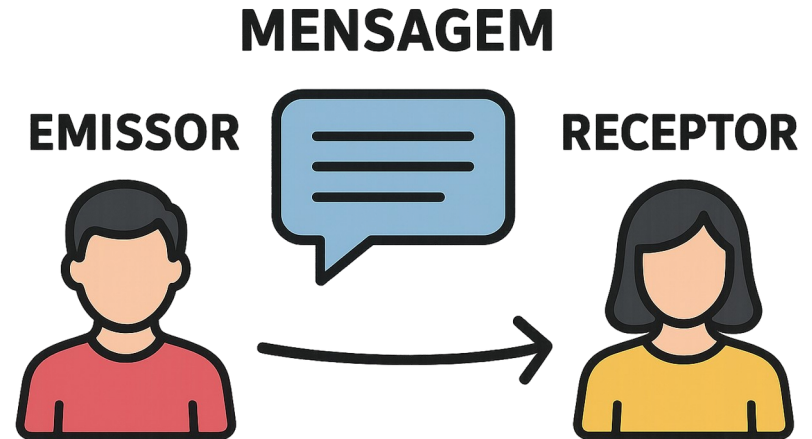
Processamento de Linguagem Natural (PLN)

Processamento de Linguagem Natural (PLN) é a área da Inteligência Artificial que busca tornar computadores capazes de compreender, interpretar e produzir linguagem humana.

Utiliza técnicas linguísticas, estatísticas, probabilísticas e de aprendizado de máquina, incluindo, atualmente, redes neurais profundas.

Uma aplicação que usa o Processamento de Linguagem Natural

tenta **imitar a comunicação dos seres humanos** e, como tal, está **sujeita aos fenômenos da comunicação humana**.



Técnicas de pré-processamento e extração de características em PLN

É a etapa de **limpeza dos dados**

Removemos pontuação, caracteres especiais, quebras de linha e tudo que puder interferir negativamente na análise textual.

- Remoção de quebras de linha
- Remoção de caracteres especiais
- Remoção de pontuação*
- Numeração*
- URLs...

Pré-processamento Textual: O Refino da Matéria-Prima

O mundo real é ruidoso. Antes de aprender, a IA precisa de dados limpos e padronizados.

Esse é um &texto de rede social :)
exemplo@hotmail.com 22231344 \n!!!



esse é um texto de rede social

</>

Prática em Python:

Ferramentas: Expressões Regulares (re), string.lower(), normalização Unicode.

Tokenização é o processo de **dividir** um texto em unidades menores, chamadas tokens, que podem ser palavras, subpalavras, caracteres ou símbolos.

Esses **tokens** são utilizados como entrada para algoritmos de Processamento de Linguagem Natural.

Esse é um texto de rede social



TOKENIZAÇÃO



Esse é um texto
de rede social

Tokenização: A Unidade Fundamental

Transformando o contínuo no discreto: as peças de lego da linguagem.

esse é um texto de rede social



```
Prática em Python:  
Bibliotecas essenciais: nltk.tokenize ou spaCy.
```

```
> █
```

Casos Extremos & Desafios

- Contrações:

[da] → [de] + [a]

- Subpalavras (BPE):
Para vocabulário infinito.

[infelizmente]



[in] + [feliz] + [mente]

Tokenização: Dividindo para Conquistar

A quebra do texto contínuo nas menores unidades de processamento: os Tokens.

1. A Sentença Normalizada

além disso, a produção será descontinuada em 8,3%.

2. A Ferramenta de Corte



Regra geral: Separar por espaços em branco e isolar pontuações.

3. O Resultado: 11 Tokens Distintos

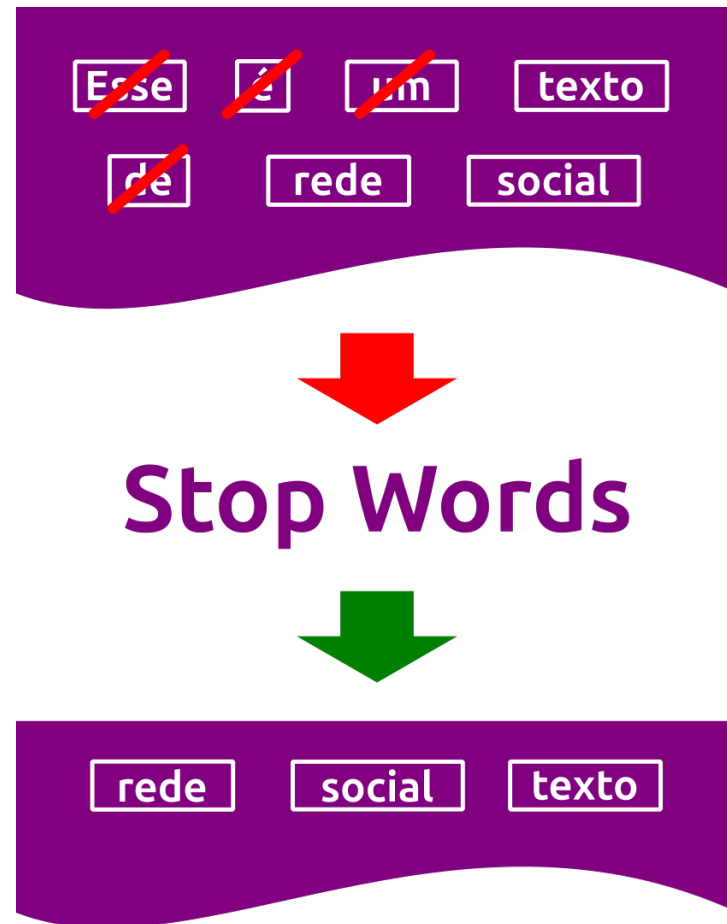


O que é um Token? Na maioria das vezes é uma palavra ou pontuação. Observe que o número 8,3 é tratado como um único token. Em IA moderna, tokens também podem ser subpalavras (pedaços de palavras) para lidar com vocabulário desconhecido.

Consiste na **remoção** de palavras que não influenciam significativamente no modelo, como preposições, artigos, etc...

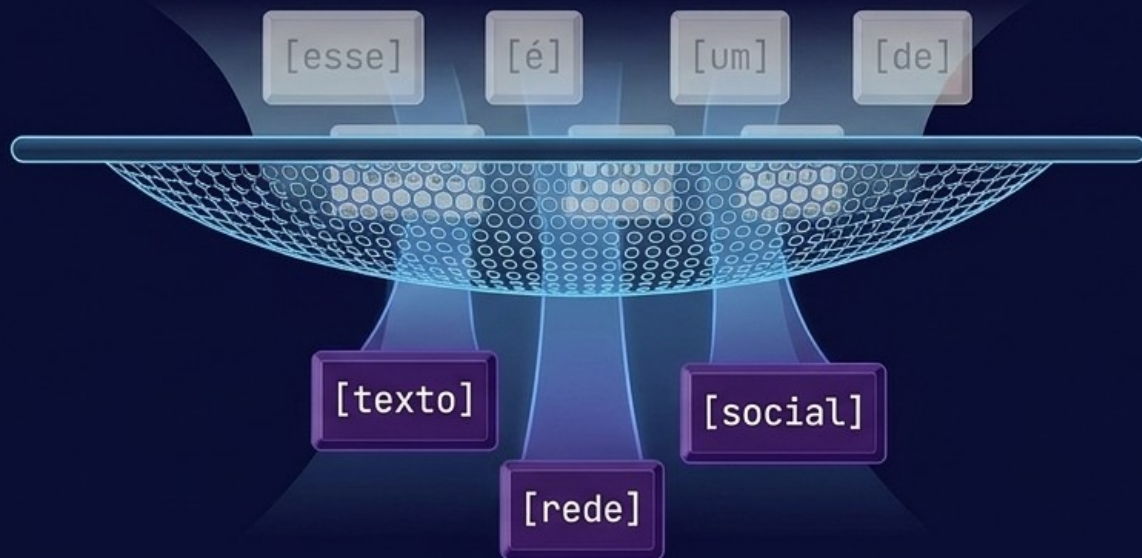
Normalmente, ocorre após a tokenização.

A biblioteca NLTK (Natural Language Tool Kit) possui uma lista de stop words para o português e outros idiomas.



Stop Words: Filtrando o Ruído Sintático

Removendo palavras de alta frequência estrutural e baixo valor semântico.



```
Prática em Python:  
Importando a lista padrão:  
nltk.corpus.stopwords.words('portuguese')
```

Stop Words: Filtrando o Ruído Gramatical

Removendo as palavras estruturais de alta frequência, mas de baixo valor semântico.

Stop Words são palavras de altíssima frequência (artigos, preposições, conjunções) descartadas para reduzir o processamento computacional.

1. Texto Tokenizado Original

[ser] [ou] [não] [ser] [,] [eis] [a] [questão]

2. Processo de Filtragem

~~[ser]~~ ~~[ou]~~ ~~[não]~~ ~~[ser]~~ [,] ~~[eis]~~ ~~[a]~~ [questão]

3. Texto Final Sobrevivente

[,] [questão]



Alerta Crítico: A remoção de Stop Words é excelente para buscas simples, mas destrói o significado em modelos avançados. Acima, a crise existencial de Hamlet foi reduzida a apenas uma 'questão'.

O **one-hot encoding** é uma técnica de codificação utilizada para transformar dados categóricos (como palavras de um texto) em representações numéricas binárias.

Nessa abordagem, constrói-se um vetor com dimensão correspondente ao número total de elementos ou palavras do vocabulário, no qual exatamente uma única posição recebe o valor 1 para indicar a presença do elemento específico, enquanto todas as outras posições permanecem preenchidas com o valor 0.

One-Hot Encoding: A Representação Binária

Identidade absoluta para cada palavra, mas total cegueira semântica.

	Maria	Está	Ouvindo	Música
Maria	1	0	0	0
Está	0	1	0	0
Ouvindo	0	0	1	0
Música	0	0	0	1



A Maldição da Dimensionalidade

Em um vocabulário real de 100.000 palavras, cada token vira um vetor com 99.999 zeros inúteis e apenas um '1'. O computador gasta memória processando o vazio.

[0, 0, 0, 0, 0, 0, 1]

Prática em Python:

Implementação rápida: `pd.get_dummies()` do Pandas ou `OneHotEncoder` do Scikit-Learn.

É uma técnica para **representar textos numericamente**.

Consiste em criar um **vocabulário** de palavras **únicas**, baseado na frequência de cada palavra.

O resultado é um **vetor de frequências** que representa o documento.

Esse é um texto de rede social

Tokenização

[Esse, é, um, texto, de, rede, social, e, é, social]

Stop words

[texto, rede, social, social]

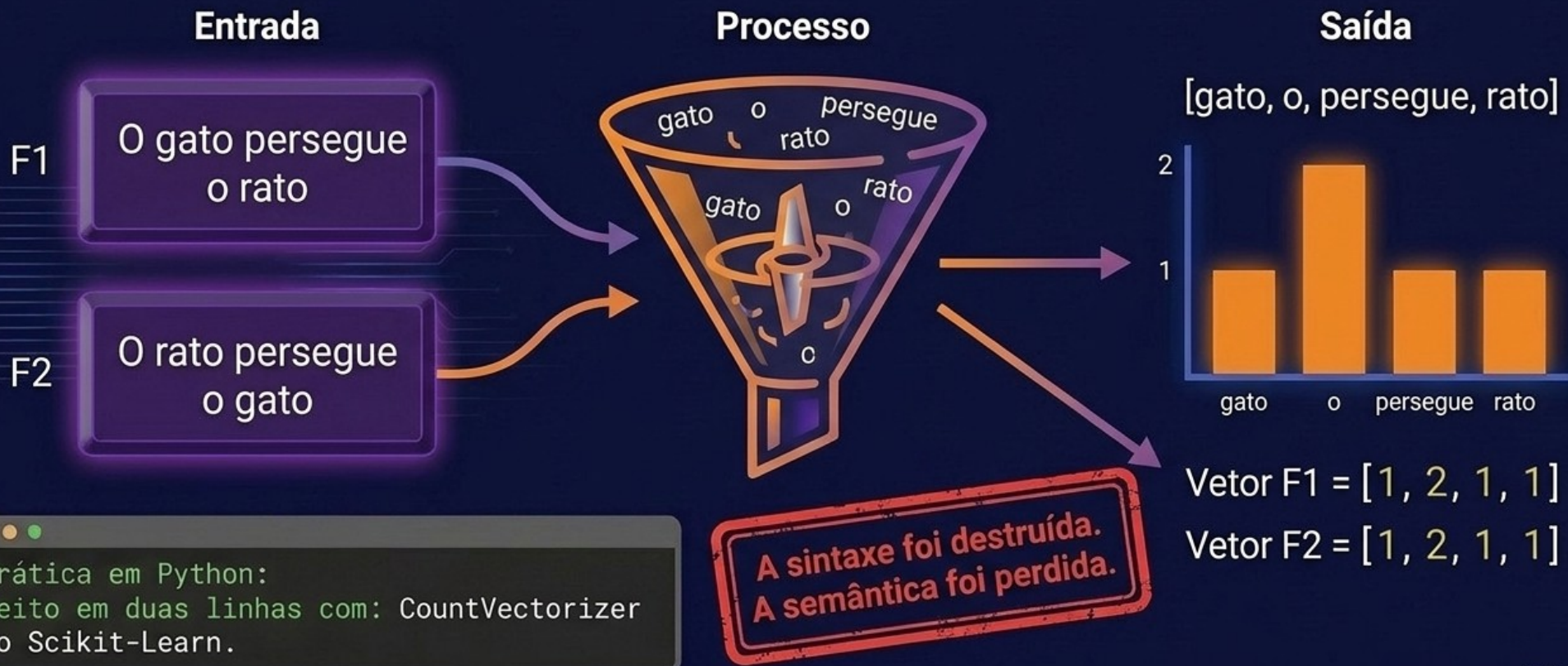
Bag of Words

texto 1 rede 1 social 2

bow=[1,1,2]

Bag of Words (BoW): O Saco de Palavras

Representando um documento inteiro pela pura contagem de seus tokens.



Bag of Words (BoW): O 'Saco de Palavras'

Convertendo tokens em números através da contagem de frequências, mas perdendo a ordem.

1. A Abordagem Matemática Básica

Ignoramos a gramática e tratamos o texto como um "saco" onde apenas a quantidade de cada palavra importa.

2. O Problema Fatal da Perda de Sintaxe

Frase A: João é mais velho do que José

Frase B: José é mais velho do que João

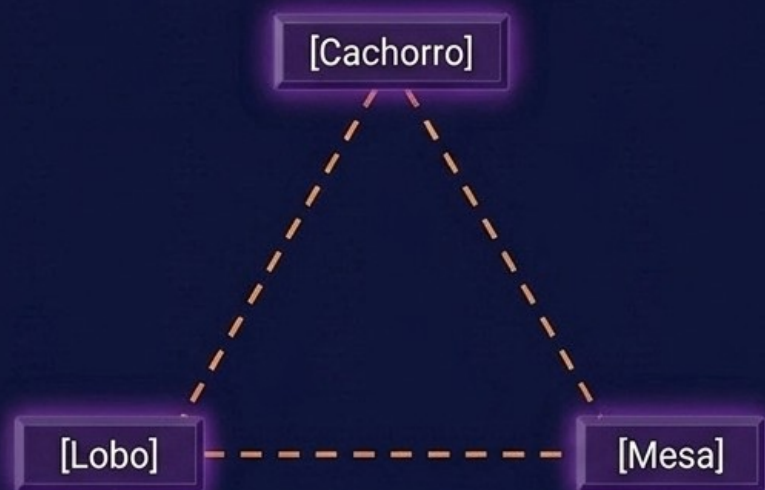
Documento (Vetor)	joão	josé	é	mais	velho	do	que
Vetor da Frase A	1	1	1	1	1	1	1
Vetor da Frase B	1	1	1	1	1	1	1

A Conclusão da Máquina: Para o computador usando BoW, as duas representações matemáticas são idênticas [1, 1, 1, 1, 1, 1, 1]. O modelo é incapaz de perceber que as frases possuem significados totalmente opostos!

O Problema da Distância Semântica

Por que vetores esparsos falham em entender o significado real?

A Falha Geométrica



Na matemática do One-Hot, todos os vetores são ortogonais. Para a máquina, a distância entre 'Cachorro' e 'Lobo' é exatamente a mesma que entre "Cachorro" e "Mesa".

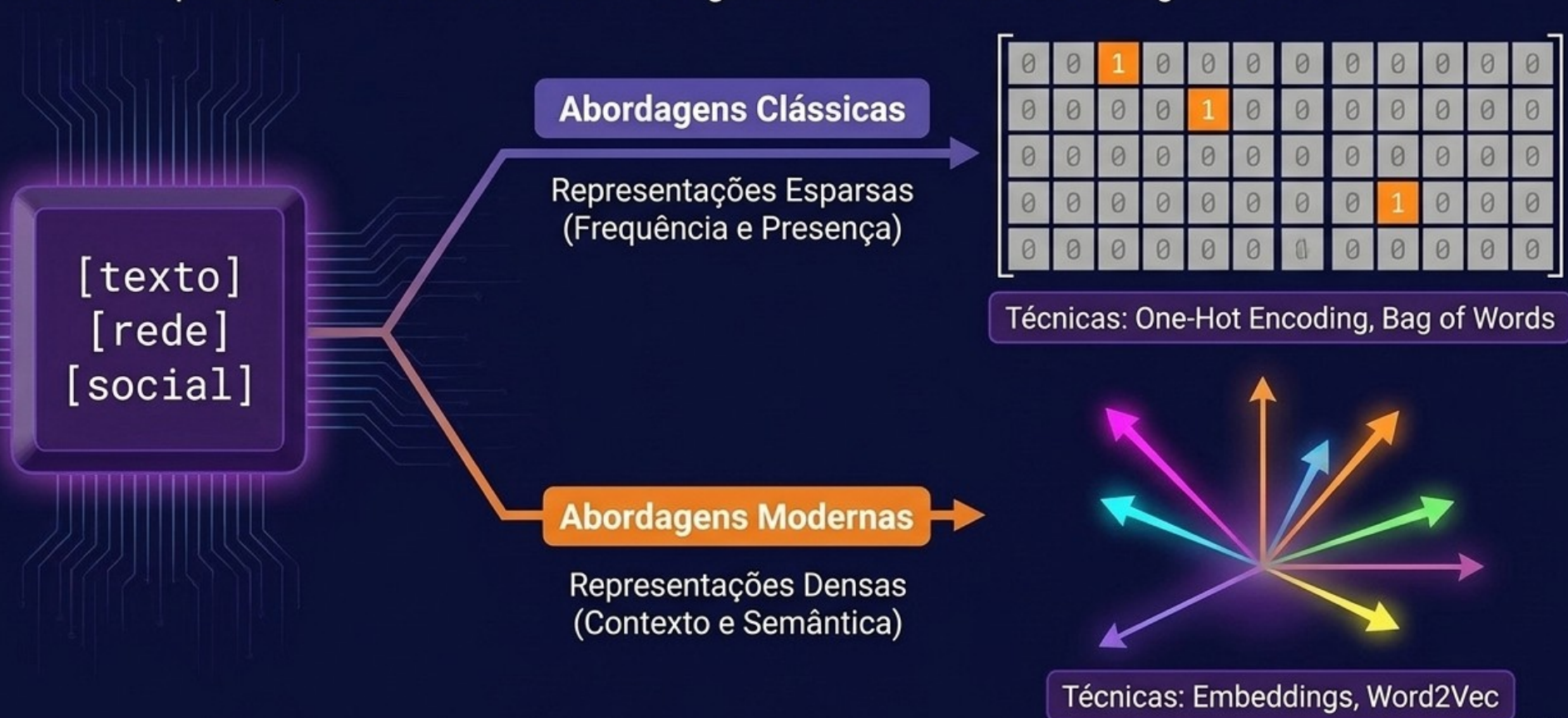
A Hipótese Distribucional

Você conhecerá uma palavra pelas companhias que ela mantém.
(J.R. Firth, 1957)

A IA não precisa de um dicionário; ela precisa mapear o contexto. Palavras que habitam os mesmos vizinhos nas frases compartilham o mesmo significado. Precisamos de vetores densos.

A Fronteira Matemática: Texto → Números

O texto está polido, mas redes neurais são cegas a letras. Entramos na álgebra linear.



Embeddings: O Salto para a Semântica

A revolução da IA: substituindo vetores vazios por vetores densos que capturam o verdadeiro significado.

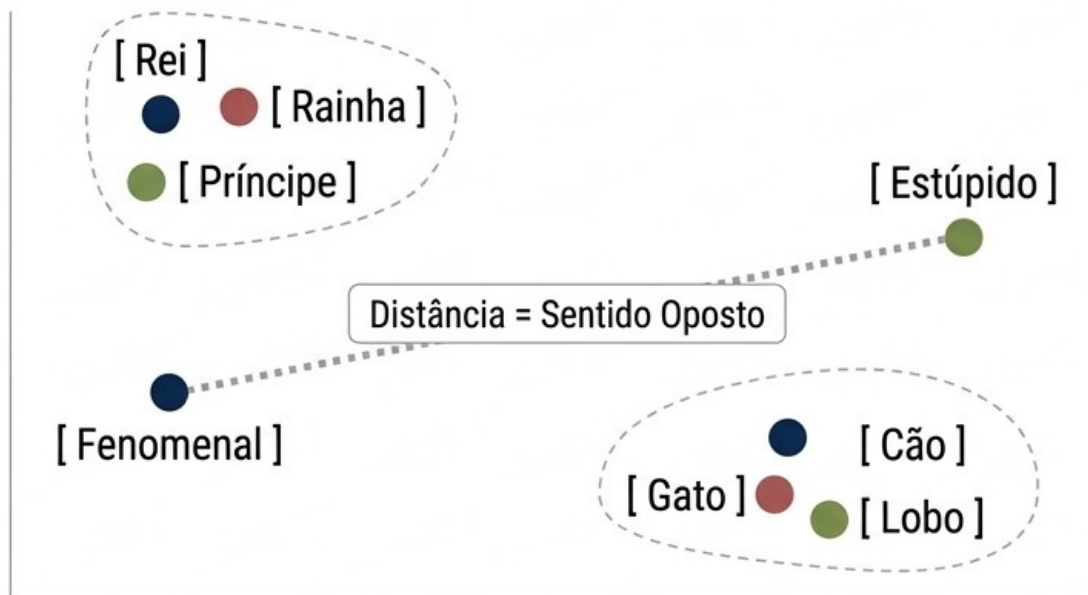
A Diferença Fundamental

Ontem (Vetores Esparsos):

Gigantes, cheios de ZEROS, sem noção de significado.

Hoje (Vetores Densos / Embeddings):

Curtos (50 a 1000 dimensões), cheios de NÚMEROS DECIMAIS. Cada dimensão representa um traço semântico (polaridade, gênero, contexto).

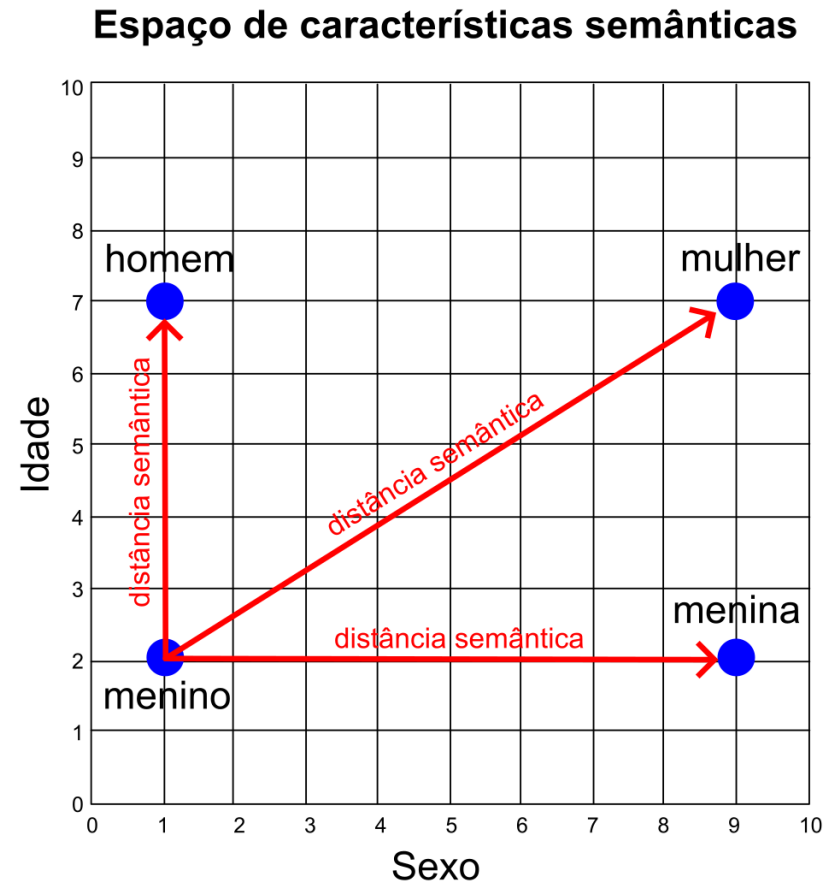


A Mágica Matemática: Palavras com significados semelhantes são mapeadas para coordenadas próximas no espaço vetorial. A máquina substitui a 'compreensão' humana por cálculos de distância geométrica.

O **Word2Vec** analisa os padrões de ocorrência (padrões de **coocorrência**) das palavras em seus contextos.

A partir desses padrões, aprende vetores numéricos que aproximam palavras de significado semelhante e afastam palavras com significados diferentes, criando um **espaço semântico** onde a distância entre os vetores representa o grau de **similaridade** entre as palavras.

É o que chamamos de **Word Embedding**



Embeddings: A Geometria do Sentido

Compactando o significado em vetores densos e multidimensionais.

One-Hot Esparso

[0, 0, 0, 0, 1, 0, 0...]

Vetor Denso (Ex: 300 dimensões)

[0.21, -0.84, 0.55, 0.12...]

Palavras com significados semelhantes agem como ímãs, atraindo-se no espaço matemático.

Família/Realeza

Rei
Rainha
Príncipe

Veículos

Moto
Carro
Caminhão

Animais

Cachorro
Gato
Lobo

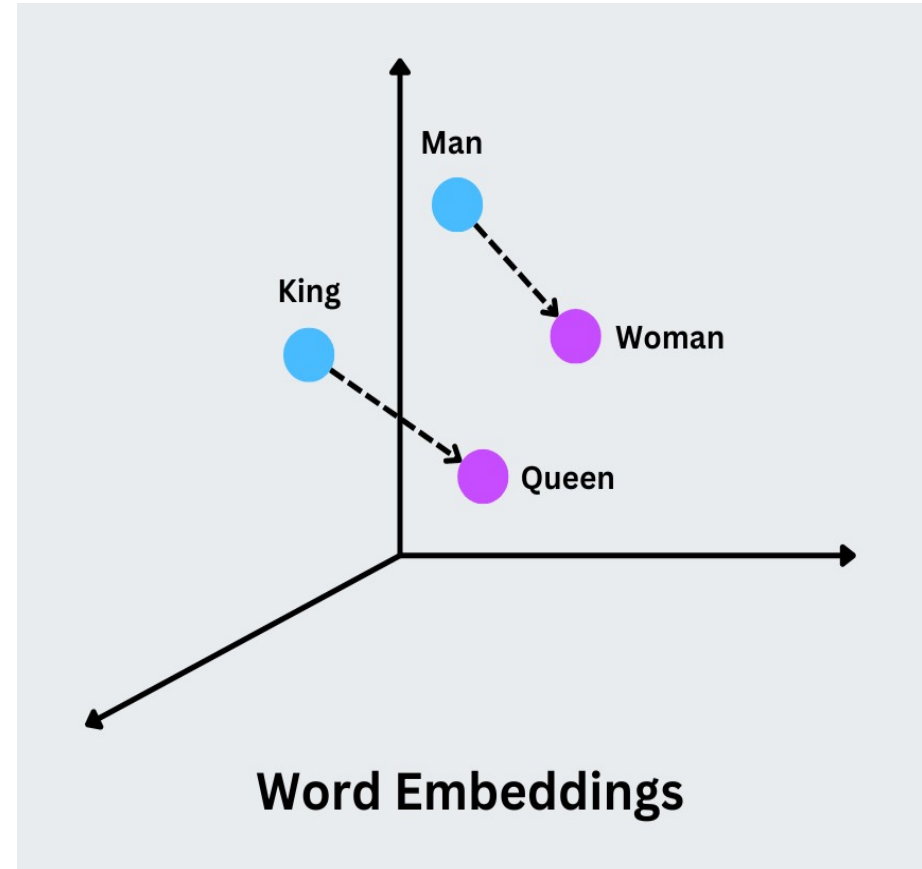
Prática em Python:

A mágica acontece na Camada de Embedding em redes neurais (ex: Embedding Layer do Keras/TensorFlow).

O **Word2Vec** analisa os padrões de ocorrência (padrões de **coocorrência**) das palavras em seus contextos.

A partir desses padrões, aprende vetores numéricos que aproximam palavras de significado semelhante e afastam palavras com significados diferentes, criando um **espaço semântico** onde a distância entre os vetores representa o grau de **similaridade** entre as palavras.

É o que chamamos de **Word Embedding**



https://github.com/nobertomaciel/PLN_Unifacs

- Autocompletar com RNN
- Análise de sentimentos
 - Tradução de textos
- Reconhecimento de fala

- [1] ALSALEH, Ahmad; PERKGOZ, Cahit. A space and time efficient convolutional neural network for age group estimation from facial images. PeerJ Computer Science, v. 9, p. e1395, 2023.
- [2] Haykin, Simon S. "Neural networks and learning machines/Simon Haykin." (2009). <https://app.minhabiblioteca.com.br/reader/books/9788577800865/pageid/0>
- [3] FACELI, Katti; LORENA, Ana C.; GAMA, João; AL, et. Inteligência Artificial - Uma Abordagem de Aprendizado de Máquina. 2. ed. Rio de Janeiro: LTC, 2021. <https://app.minhabiblioteca.com.br/reader/books/9788521637509/>.
- [4] <https://www.datacamp.com/pt/tutorial/tutorial-for-recurrent-neural-network>
- [5] <https://www.ibm.com/br-pt/think/topics/recurrent-neural-networks>
- [6] <https://pessoas.unifal-mg.edu.br/nelsonlemes/neuronio-de-mcculloch-pitts/>
- [7] <https://www.deeplearningbook.com.br/introducao-as-redes-neurais-convolucionais/>
- [8] <https://medium.com/neuronio-br/entendendo-redes-convolucionais-cnns-d10359f21184>
- [9] CASELI, Helena de Medeiros; NUNES, Maria das Graças Volpe (org.). Processamento de Linguagem Natural: conceitos, técnicas e aplicações em português. 2. ed. São Carlos: BPLN, 2024. Disponível em: https://brasileiraspln.com/livro-pln/2a-edicao/Livro_PLN_BPLN_3ed.pdf